

JavaScript DOM Manipulation

Deep Dive



What is the DOM?

The DOM (Document Object Model) is like a living blueprint of a webpage. JavaScript uses it to find, change, and interact with parts of the page!

Real-Life Analogy

Rangoli pattern on the floor

Cricket scoreboard

School notice board

DOM Equivalent

Structure of the webpage (HTML elements)

Changing content dynamically (innerHTML)

Adding/removing elements on the page



The DOM Tree

The DOM organises HTML like a family tree.
Each element is a node with parents, children, and siblings.



```
<html>          <-- Root (Grandmother)
  <head>          <-- Child of html
    <title>My Page</title>
  </head>
  <body>          <-- Child of html (Sibling of head)
    <h1>Hello!</h1>    <-- Child of body
    <p>Welcome</p>      <-- Sibling of h1
  </body>
</html>
```

Root = top element | Parent = container | Child = inside | Sibling = same level

JavaScript - The Helper

JavaScript is the behaviour layer of a webpage.
It makes pages interactive - like a helper friend who does things for you!

- ✓ Find parts of the page (like finding a student by roll number)
- ✓ Change text or content (like updating a scoreboard)
- ✓ React to clicks and events (like a doorbell ringing)
- ✓ Add or remove elements (like pinning notes on a board)



Selecting Elements - getElementById()

Every HTML element can have a unique ID - like a school roll number or Aadhaar card.
getElementById() finds that exact element!

```
// HTML:  
<h1 id="myHeading">Hello World</h1>  
  
// JavaScript:  
let heading = document.getElementById('myHeading');  
console.log(heading.innerHTML); // Output: Hello World
```

Common Mistakes

- It is case-sensitive: getElementById (not getElementByID)
- Always use quotes around the ID name
- Spelling must match exactly with the HTML id



Changing Text - innerHTML

innerHTML lets you read or change the text inside any HTML element.
Think of it like changing the menu at a chai shop!

```
// Reading innerHTML:  
let heading = document.getElementById('myHeading');  
console.log(heading.innerHTML); // Output: Hello World  
  
// Changing innerHTML:  
heading.innerHTML = 'Namaste, India!';  
console.log(heading.innerHTML); // Output: Namaste, India!
```

.innerHTML vs .textContent



onclick Events

An event is something that happens - like ringing a doorbell!
onclick runs code when a user clicks a button.



Method 1: Inline onclick

```
<button onclick="changeHeading()">Badlo!</button>

<script>
  function changeHeading() {
    document.getElementById('myHeading').innerHTML = 'Namaste, India!';
  }
</script>
```

Method 2: addEventListener

```
document.getElementById('myBtn').addEventListener('click', changeHeading);

function changeHeading() {
  document.getElementById('myHeading').innerHTML = 'Namaste, India!';
}
```

Writing Functions in <script>

Functions are reusable blocks of code. We write them inside <script> tags.
They only run when called!

```
<!DOCTYPE html>
<html>
<body>
  <h1 id="myHeading">Hello World</h1>
  <button onclick="changeHeading()">Click Me!</button>

  <script>
    function changeHeading() {
      document.getElementById('myHeading').innerHTML = 'Namaste!';
    }
  </script>
</body>
</html>
```

Key Rules

- JavaScript is case-sensitive (myHeading is not myheading)
- Always end statements with a semicolon ;
- Use curly braces { } to wrap function code



addEventListener - A Cleaner Way

addEventListener separates HTML from JavaScript - keeping code clean and organised.
It listens for an event and runs a function when it happens.

```
// Syntax:  
element.addEventListener('event', functionName);  
  
// Example:  
let btn = document.getElementById('myBtn');  
btn.addEventListener('click', function() {  
    document.getElementById('myHeading').innerHTML = 'You clicked!';  
});
```

Why use addEventListener?

- Keeps HTML clean (no onclick in the tag)
- You can add multiple listeners to one element
- Easier to read and maintain for bigger projects



Full Flow Recap

What happens when you click a button that changes text?

Step 1: User clicks the button

Step 2: onclick event fires

Step 3: JavaScript function runs

Step 4: getElementById() finds the element

Step 5: innerHTML changes the text

Click Button --> Find Element --> Change Text

```
// The complete flow in code:  
button.onclick = function() {  
  document.getElementById('myHeading').innerHTML = 'New Text!';  
};
```



Check-in Quiz

Q1: What does DOM stand for?

Answer: `Document Object Model`

Q2: Which method finds an element by its ID?

Answer: `document.getElementById()`

Q3: What does `.innerHTML` do?

Answer: Reads or changes the content inside an element

Q4: What event fires when you click a button?

Answer: `onclick` / `click` event

Q5: Where do we write JavaScript in HTML?

Answer: Inside `<script>` tags



Hands-On Activity

Create a webpage that changes a heading to YOUR name when you click a button!

My Magic Name Board

```
<!DOCTYPE html>
<html>
<head>
  <title>My Magic Name Board</title>
</head>
<body>
  <h1 id="nameBoard">Virat Kohli</h1>
  <button onclick="changeName()">Show My Name!</button>

  <script>
    function changeName() {
      document.getElementById('nameBoard').innerHTML = 'Arjun';
    }
  </script>
</body>
</html>
```

Steps:

1. Open Notepad or VS Code
2. Type the code above exactly
3. Save as magic-name.html
4. Open in your browser
5. Click the button and watch the magic!



Key Takeaways

Quick Reference Card



```
// 1. Select an element:
let el = document.getElementById('myId');

// 2. Read its content:
console.log(el.innerHTML);

// 3. Change its content:
el.innerHTML = 'New text here!';

// 4. React to a click:
el.onclick = function() {
  el.innerHTML = 'You clicked me!';
};
```

Remember:

- DOM = the living blueprint of your webpage
- `getElementById()` = find any element by its unique ID
- `innerHTML` = read or change text inside elements
- `onclick` = make things happen when users click
- `<script>` tags = where JavaScript lives in HTML

Thank You!

Ready to make magic webpages?

Keep practising and experimenting with code.

Ask questions - that is how coders learn!

